DPU-BSTU Joint Institute

# COURSE PROJECT

# EXPLANATORY NOTE

Course name: Scripting Programming Languages
Topic: Development of a Web Application for a Music Store

**Student**
2nd year
Group 803977 _____ John Doe
signature

**Supervisor**
Assistant _____ Veronica Aleshauskas
position, academic degree, academic title — signature

Grade received: _____

Supervisor _____ _____ Veronica Aleshauskas
signature — date — name and surname

Dalian 2026

## Content

Introduction ..... 4
1 Design of the Web Application ..... 5
1.1 Similar Solutions ..... 5
1.2 Problem Statement ..... 8
2 Development of the Web Application ..... 10
3 Testing of the Web Application ..... 11
Conclusion ..... 12
Appendix A Web Application screenshots ..... 13
Appendix B Search functionality listing ..... 14
Appendix C Data verification listing ..... 15

# Introduction

In the modern digital economy, the music industry has undergone a profound transformation. Physical sales have given way to streaming platforms and digital storefronts, yet the demand for curated, user-friendly online music stores remains strong. Today, a music store website is far more than a static catalog: it is a dynamic web application that enables users to explore artist discographies, preview audio tracks, discover new releases, access exclusive content, and purchase digital or physical media securely and conveniently.

The development of such a web application presents a unique intersection of technical implementation, user experience design, and business strategy. A successful music store must offer an intuitive and visually appealing interface, fast and reliable performance, and robust functionality that meets the expectations of modern users. Particular attention must be paid to responsive design, as a growing number of visitors access music stores via smartphones and tablets. Furthermore, the application must be cross-browser compatible to ensure consistent operation across all major web browsers.

This coursework is dedicated to the development of a fully functional music store web application using modern front-end technologies. The project is guided by specific technical constraints and requirements, which define both the architecture and the implementation approach.

The goal of this coursework is to develop a music store web application using JavaScript, adhering to the ECMAScript 9 (ES2018) specification and incorporating essential e-commerce features.

To achieve this goal, the following objectives are set:

1. To analyze existing markup languages, tools, libraries, and frameworks used in modern web development, with a focus on client-side scripting.
2. To design the application structure and requirements informed by user experience principles, including a description of technical capabilities.
3. To develop the core functional components, including:
   - a product catalog with sorting functionality based on various criteria (e.g., by genre, artist, price, or release date);
   - a user interaction form featuring input validation and default value substitution;
   - event-driven mechanisms for dynamic style changes and function triggering.
4. To populate the website with relevant music-related content (artist profiles, album data, track listings, pricing).
5. To ensure the web application is responsive and cross-browser compatible.
6. To test the web application for functionality, usability, and compatibility across different devices and browsers.

This structure allows for a systematic presentation of the entire development lifecycle: from initial analysis and design through implementation, testing, and final evaluation. The following sections will provide a detailed account of each stage, with particular emphasis on the JavaScript implementation, which constitutes the core of the application's interactive functionality.

# 1 Design of the Web Application

To develop a competitive and user-friendly music store web application, it is essential to analyze existing solutions in the market. This analysis helps identify common design patterns, functional strengths, and typical drawbacks that can be avoided in the developed project.

## 1.1 Similar Solutions

BeatPort (www.beatport.com) is one of the largest digital music stores, specializing in electronic music. The website features a clean, dark-themed interface that emphasizes album artwork and track listings.

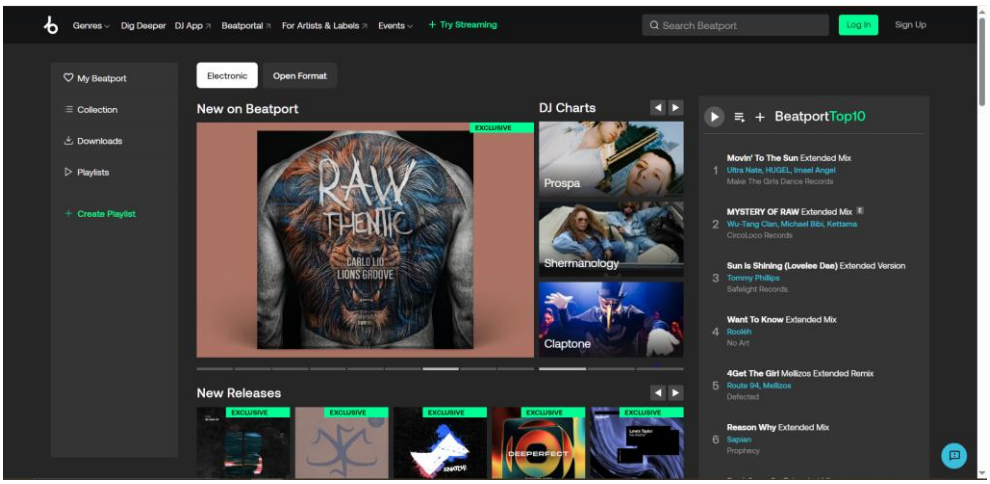



Figure 1.1 – Home page of the “BeatPort” website

One noticeable drawback is the overly dense information layout. The homepage contains too many sections: charts, new releases, recommended tracks, and curated playlists. This can overwhelm visitors, especially first-time users who may struggle to locate specific content. Additionally, the navigation menu is extensive and somewhat cluttered, making it difficult to quickly switch between genres or find artist pages.

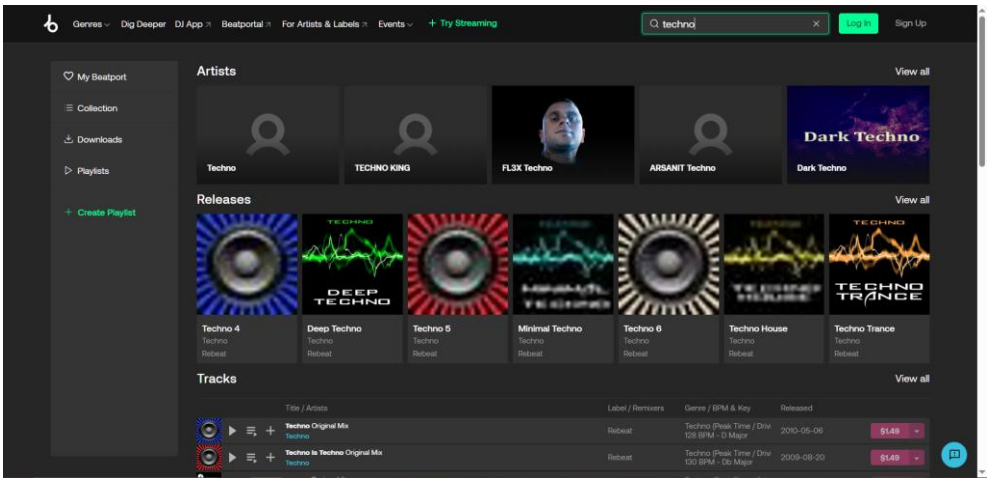



Figure 1.2 – Search page of the “BeatPort” website

On the other hand, it is worth noting the powerful filtering and sorting system, which allows users to browse tracks by genre, tempo, key, and release date. This functionality greatly enhances the user experience for professional DJs and producers. The audio preview feature is also implemented effectively, enabling users to listen to track samples before making a purchase.

Bandcamp (https://bandcamp.com/) takes a different approach, focusing on direct artist-to-fan sales. The primary advantage of Bandcamp is its artist-centric design, where each artist has a dedicated page with their discography, biography, and merchandise.

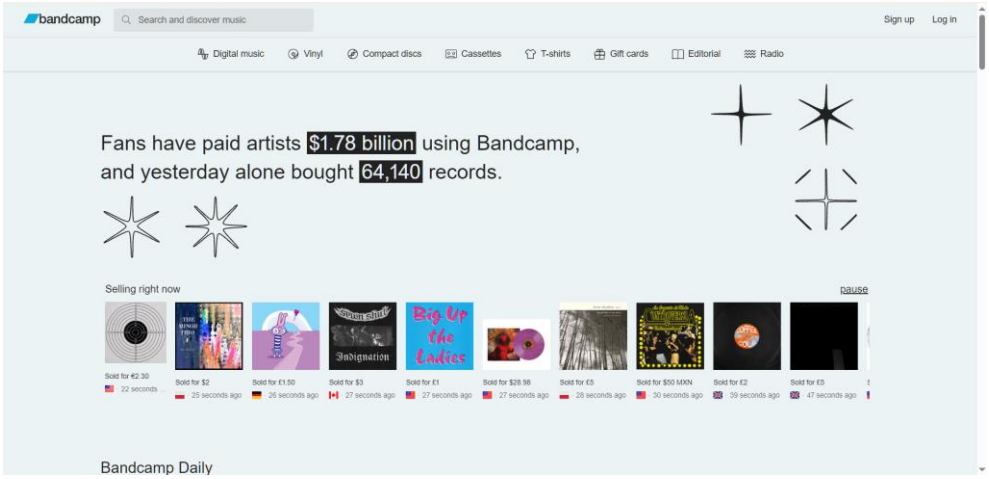



Figure 1.3 – Home page of the “Bandcamp” website

However, a significant drawback of this site is the lack of a unified catalog structure. Because each artist page is independently customized, the overall browsing experience can be inconsistent. Some pages are well-organized, while others appear chaotic or incomplete.

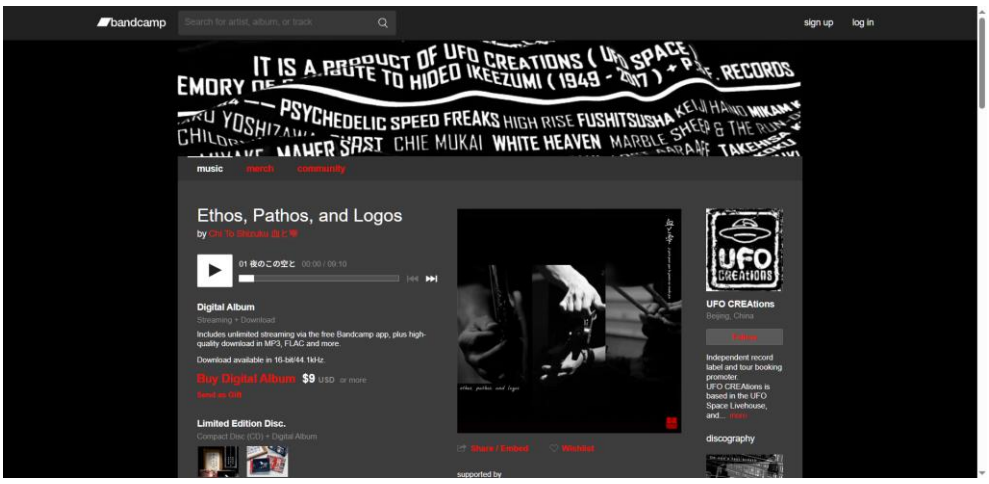



Figure 1.4 – Artist page of the “Bandcamp” website

Furthermore, the search functionality is limited; users cannot filter by price range, release year, or format (digital vs. physical), which complicates the discovery process.

On individual artist pages, users can view albums, listen to full tracks (with a limited number of free plays), and purchase music in various formats. Among the downsides, I can

note that the player interface is minimal and lacks advanced controls such as playlist creation or queue management.

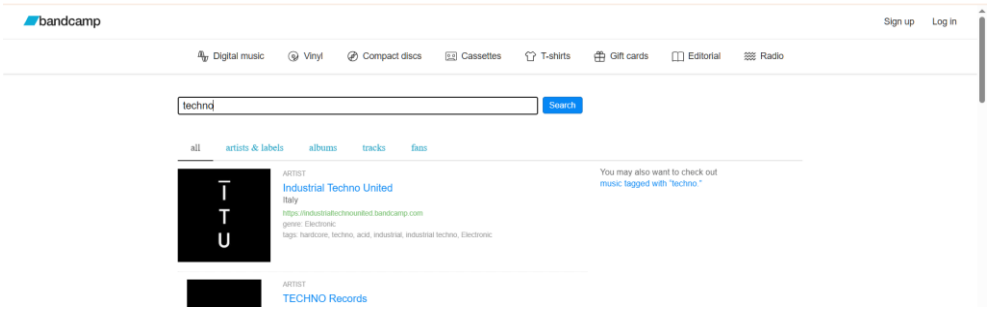



Figure 1.5 – Search page of the “Bandcamp” website

The next platform I considered is Apple Music (music.apple.com). After visiting the main page, I was redirected to a promotional landing page rather than a direct store interface. Apple Music's web presence primarily serves as a marketing portal for its subscription service.

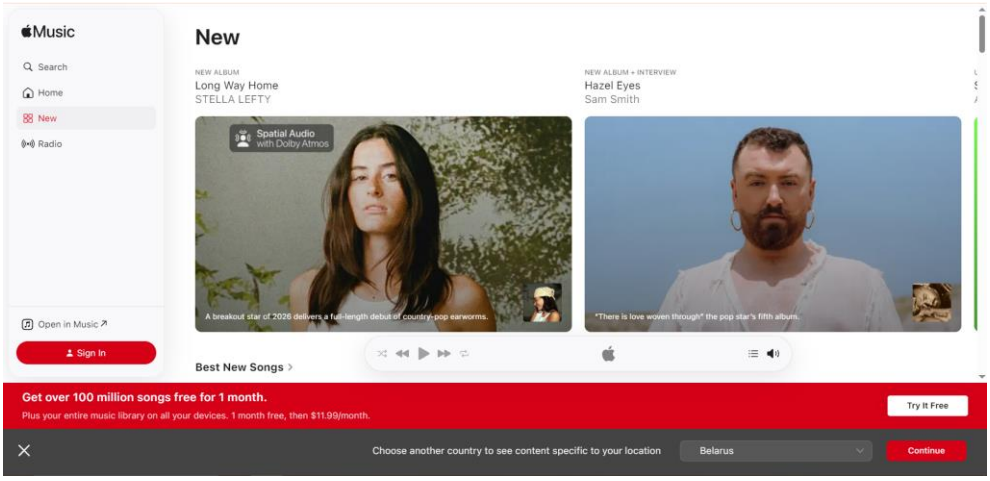



Figure 1.6 – Home page of the “Apple Music” website

The landing page features a minimalist design with large, high-quality visuals of featured artists and curated playlists. The aesthetic is polished and professional, but the lack of direct purchasing options (since Apple Music operates on a subscription model rather than a per-track store) makes it less relevant as an example for this project.

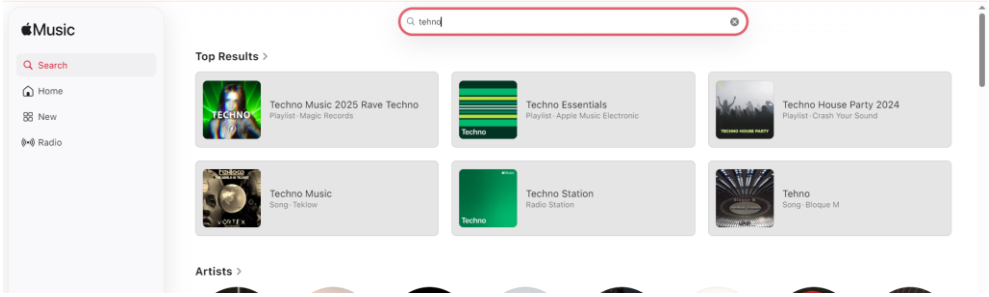



Figure 1.7 – Search page of the “Apple Music” website

Other disadvantages include the limited navigation: users cannot browse by genre or label without first subscribing.

Based on the results of the analysis of similar solutions, it is possible to formulate requirements for the website being developed.

## 1.2 Problem Statement

This project requires the creation of a music store web application consisting of three to four pages. The website should contain general information about the music store, including contact details, a photo gallery of artists or merchandise, store location information, a description of the store's mission and offerings, and a product catalog. The primary content consists of text, album artwork, artist photographs, and audio preview functionality.

The application will use HTML5 for page structure, CSS3 for styling, and JavaScript in accordance with the ECMAScript 9 (ES2018) specification for interactive functionality. The website must be responsive, cross-browser compatible, and include the following functional components implemented via JavaScript.

At the top of each page, there will be a header containing:

1. The store's logo and name.

2. A horizontal navigation menu with links to other pages: Home, Catalog, Artists, About Us, Registration, and Contacts.

The navigation menu will be consistent across all pages, and JavaScript will handle interactive elements such as dropdown menus, mobile hamburger menu toggles, and active page highlighting. On the mobile version (up to 768px wide), a JavaScript-driven hamburger menu will toggle the visibility of the navigation links using classList.toggle().

Home Page contains basic information about the music store, including its mission, featured artists or albums, and promotional banners. High-quality photographs of merchandise or artist imagery will be displayed to create an engaging visual experience.

Catalog Page displays the current product catalog, including albums and individual tracks. Each entry will include the artist name, album title, genre, release year, price, and a sorting functionality based on various criteria (e.g., by price, genre, artist, or release date). Audio preview functionality will allow users to listen to track samples.

JavaScript will be used to implement sorting functionality without reloading the page. The product data will be stored as a JavaScript array of objects (containing fields such as artist, title, genre, price, and releaseYear). When a user selects a sorting option from a dropdown menu, a change event listener will trigger a function that sorts the array using Array.prototype.sort() and dynamically re-renders the product list using DOM manipulation methods such as innerHTML or appendChild(). Sorting criteria will include price, artist name, genre, and release date.

Artists Page features a list of artists represented by the store, with brief biographies and links to their respective albums. This page enhances discoverability and helps users navigate by performer.

About Us Page contains information about the store's history, mission, and team, along with contact details and links to social media.

Registration Page contains a user registration form with input validation and default value substitution. Registration will allow users to receive promotional newsletters, access exclusive content, and earn loyalty bonuses.

When the page loads, certain fields will be pre-filled (e.g., the genre dropdown will default to “All Genres”, and the newsletter checkbox will be checked by default).

On form submission, a submit event listener will prevent default behavior and execute validation functions. Validation rules will include: name (required, minimum 2 characters), email (valid format using regular expressions), password (minimum 6 characters), and password confirmation matching. Error messages will be displayed dynamically next to each field.

If all validations pass, a success message will be displayed and the user data will be stored in a JavaScript object or logged to the console.

Contacts Page features phone numbers, email addresses, and links to social media profiles.

A footer should be located at the bottom of each page, containing contact information, copyright notices, and brief additional text.

In the mobile version (up to 768px wide), pages should adapt to the device width. At a width of 768px or greater, the desktop version should be displayed. This ensures optimal viewing and interaction across all devices.

JavaScript will handle various events across all pages to change styles dynamically and trigger functions:

– a “Dark Mode” toggle button, if implemented, will add or remove a CSS class on the body element. Interactive buttons (e.g., “Add to Wishlist”) will change color, text, or icon when clicked using classList.toggle() or style property manipulation;

– audio preview buttons will trigger playback functions and change their text to “Playing... ” when clicked. Clicking on an artist or album card may trigger functions that filter the catalog or display additional information;

– all event listeners will be attached using addEventListener().

All JavaScript code must comply with the ECMAScript 9 (ES2018) specification. This includes using const and let for variable declarations, arrow functions, template literals, array destructuring, spread syntax, Array.prototype.map(), Array.prototype.sort(), classList methods, and regular expressions. The code must be clean, well-commented, and organized into logical functions. No external JavaScript libraries or frameworks will be used; all functionality must be implemented using basic JavaScript.

# 2 Development of the Web Application

# 3 Testing of the Web Application

# Conclusion

# Appendix A
# Web Application screenshots

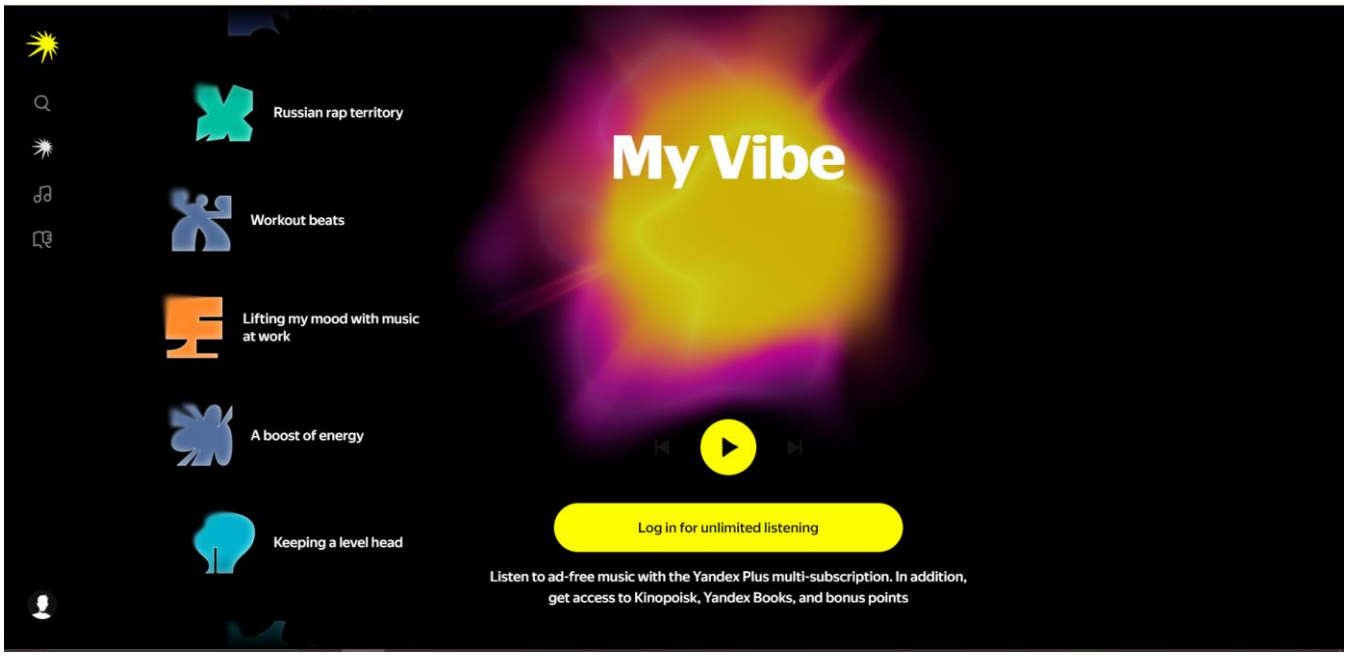



Figure A.1 – Home page

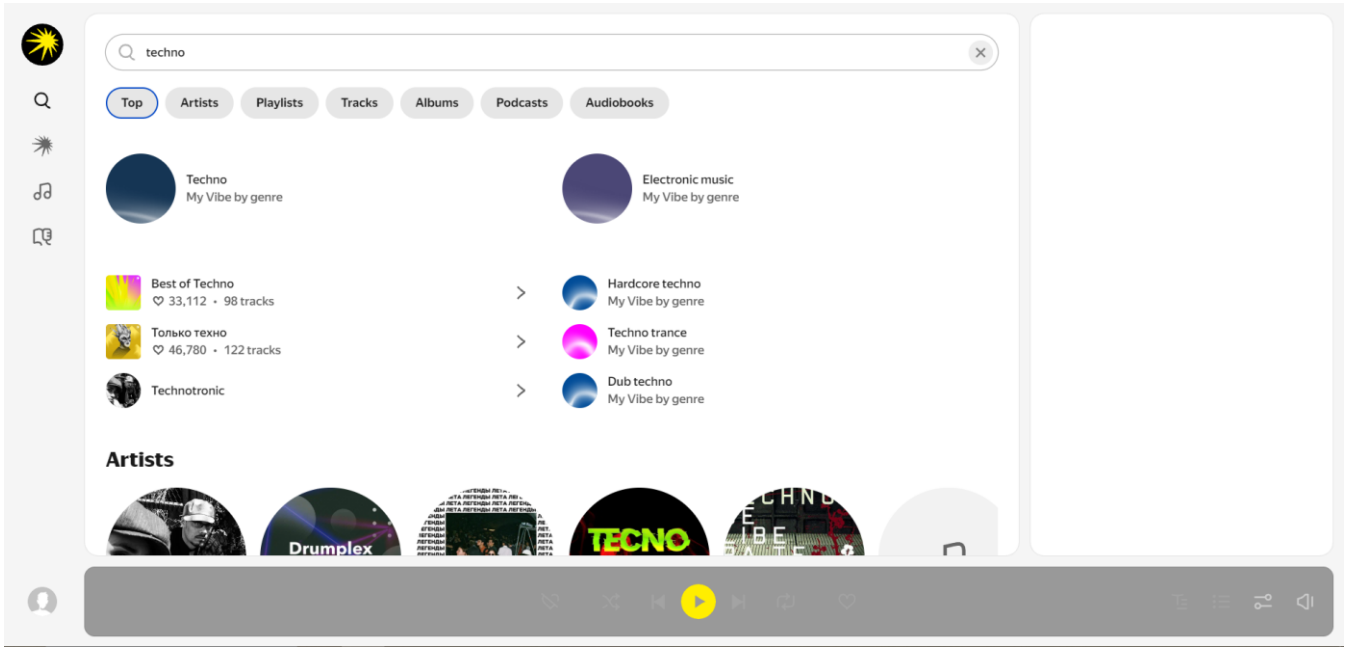



Figure A.2 – Search page

## Appendix B
## Search functionality listing

```
const products = [
    { id: 1, artist: 'The Beatles', title: 'Abbey Road', genre: 'Rock',
price: 12.99 },
    { id: 2, artist: 'Daft Punk', title: 'Random Access Memories',
genre: 'Electronic', price: 14.99 },
    // ... more products
];

const searchProducts = (query) => {
    const trimmedQuery = query.trim().toLowerCase();
    if (trimmedQuery === '') {
        renderAllProducts(); // show all products if search is empty
        return;
    }
    const results = products.filter(product =>
        product.artist.toLowerCase().includes(trimmedQuery) ||
        product.title.toLowerCase().includes(trimmedQuery) ||
        product.genre.toLowerCase().includes(trimmedQuery)
    );
    renderResults(results);
};

document.querySelector('#search-button').addEventListener('click', ()
=> {
    const query = document.querySelector('#search-input').value;
    searchProducts(query);
});

document.querySelector('#search-input').addEventListener('keyup',
(event) => {
    if (event.key === 'Enter') {
        const query = document.querySelector('#search-input').value;
        searchProducts(query);
    }
});
```

## Appendix C
## Data verification listing

```
document.addEventListener('DOMContentLoaded', () => {
    // Set default values
    document.querySelector('#genre').value = 'all';
    document.querySelector('#newsletter').checked = true;

    const form = document.querySelector('#registration-form');
    form.addEventListener('submit', (event) => {
        event.preventDefault();

        // Clear previous errors
        document.querySelectorAll('.error-message').forEach(el      =>
el.textContent = '');
        document.querySelectorAll('.form-input').forEach(el         =>
el.classList.remove('error'));

        let isValid = true;

        const name = document.querySelector('#name').value.trim();
        if (name.length < 2) {
            document.querySelector('#name-error').textContent = 'Full
name must be at least 2 characters.';
            document.querySelector('#name').classList.add('error');
            isValid = false;
        }

        const email = document.querySelector('#email').value.trim();
        const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
            document.querySelector('#email-error').textContent       =
'Please enter a valid email address.';
            document.querySelector('#email').classList.add('error');
            isValid = false;
        }

        const password = document.querySelector('#password').value;
        if (password.length < 6) {
            document.querySelector('#password-error').textContent    =
'Password must be at least 6 characters.';

document.querySelector('#password').classList.add('error');
            isValid = false;
        }

        const     confirm     =     document.querySelector('#confirm-
password').value;
        if (password !== confirm) {
            document.querySelector('#confirm-error').textContent     =
'Passwords do not match.';
```

```
            document.querySelector('#confirm-
password').classList.add('error');
            isValid = false;
        }

        if (isValid) {
            const userData = {
                name: name,
                email: email,
                genre: document.querySelector('#genre').value,
                newsletter:
document.querySelector('#newsletter').checked
            };
            console.log('Registration successful:', userData);
            document.querySelector('#form-message').innerHTML =
                '<p class="success"> Registration successful! Welcome
to the music store!</p>';
        }
    });
});
```